DATA-236 Sec 12 - Distributed Systems for Data Engineering

Uber Simulation Project

Team - 7

Group Members & Contribution:

Group Member	Contribution
Bindu Madhavi Edara (018179005)	Implemented the Rides module, including ride creation, listing, editing, and ride statistics.
Harika Eadara (018178992)	Handled the Billing module, supported dynamic pricing using ML
Hemanvitha Katakam (017660786)	Developed the Customer module, including registration, authentication**.**
Nandhakumar Apparsamy (018190003)	Built the Driver module, including registration, profile management, and driver lookup, Kafka.
Venkata Siddarth Gullipalli (017660669)	Designed and implemented the Admin module, created ML logic for pricing, and built analytics features including charts and revenue reports.

GitHub Link: https://github.com/Nandha951/DATA-236-Uber-Project

Object Management Policy

In our Uber Simulation project, we implemented a robust object management policy for five core entities: Drivers, Customers, Billing, Admins, and Rides. These services use MongoDB as the backend database. We selected it for its schema flexibility, scalability, and natural support for embedded documents such as ride history and credit card info. Each service is designed to support CRUD operations and communicates asynchronously when needed via Kafka. We ensure consistent state management, strict validation of user inputs, and extensible schema design.

Key Components:

- **Schema Definition and Validation**
  - Strict schema definitions for all entities (Customer, Driver, Ride)
  - Built-in validation for required fields
  - Custom validation for data formats (SSN, email, phone numbers)

    - State names: Limited to valid U.S. state codes or full names
    - Range validation for ratings (1-5)
- **Data Integrity:**
    - Unique constraints on critical fields (SSN, email)
    - Referential integrity through MongoDB ObjectIds
    - Automatic timestamp tracking (createdAt, updatedAt)
    - Indexing on frequently queried fields
    - Passwords encrypted using `bcryptjs`
    - Sensitive data (e.g., credit cards) is stored securely via a nested schema
- **Error Handling**
    - Validation errors are caught and handled appropriately
    - Custom error messages for a better user experience
    - Proper error propagation through the service layers

Handling Heavyweight Resources

Our application handles heavyweight resources through a microservices architecture, ensuring scalability and maintainability. In a distributed microservices architecture, certain resources, such as database connections, external APIs, and in-memory caches, are considered heavyweights due to their high performance and system memory costs.

Database Connections

- Each microservice uses a connection pool with Mongoose (for MongoDB) to avoid repeatedly opening and closing connections.
- Connection reuse ensures efficient memory and network usage.

Redis Caching

- We use Redis to cache frequent queries like:
    - Driver list by location
    - Ride history lookups

This reduces database read pressure and improves frontend responsiveness.

Kafka Messaging

- Kafka acts as a lightweight message broker to decouple services.
- For example, the Ride Service sends booking messages to Kafka that the Billing Service picks up, keeping operations asynchronous and scalable.

Implementation Details:

- **Service Architecture**

    - Customer Service
    - Driver Service
    - Ride Service
    - Billing Service
    - Admin Service
- **Resource Management**
    - Containerized services using Docker
    - Dedicated upload directory for file handling
    - Asynchronous processing for long-running tasks
    - Proper connection pooling for database access
- **Scalability Considerations**
    - Independent scaling of services
    - Load balancing capabilities
    - Efficient resource utilization
    - Proper error handling and recovery

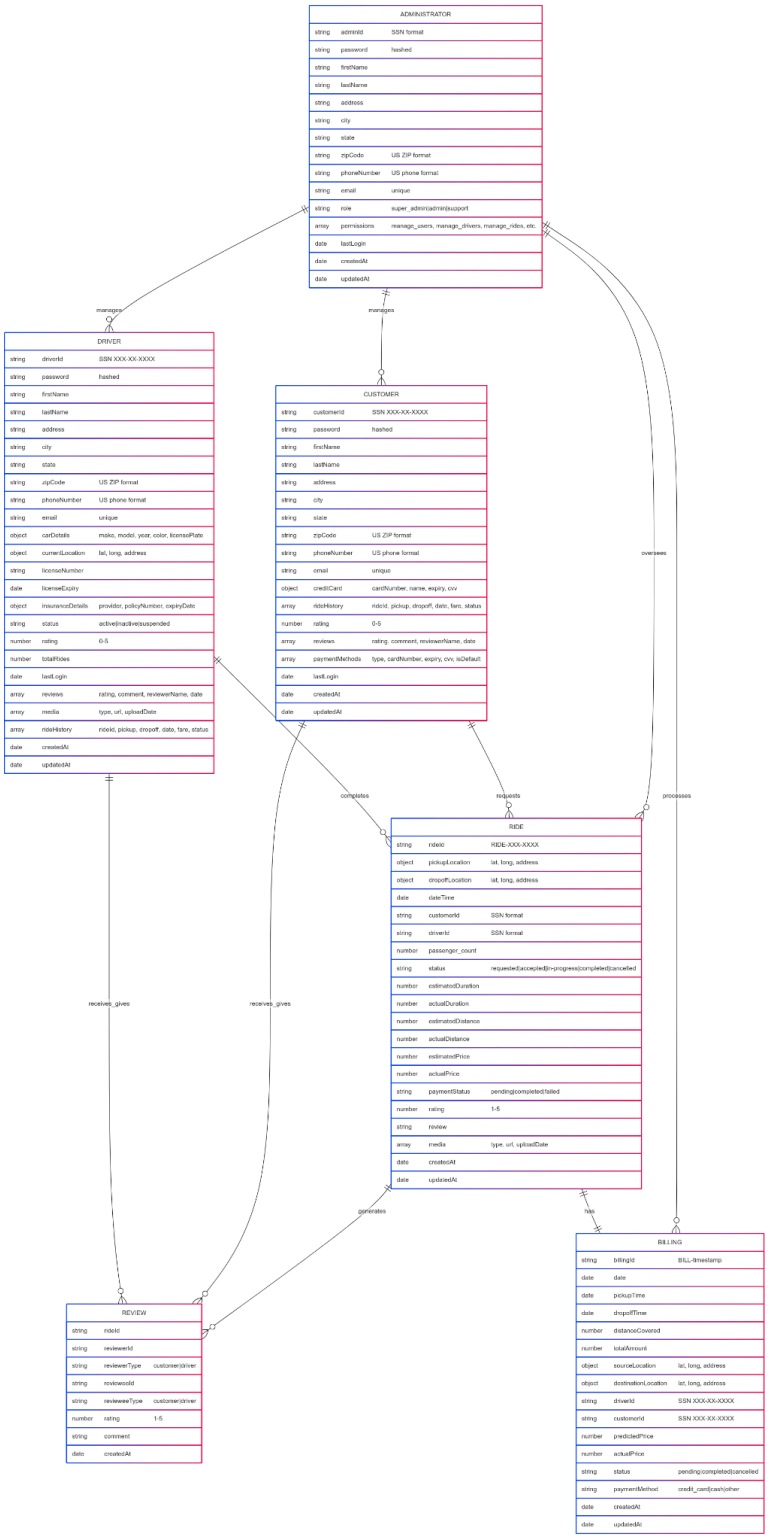
C. Database Write Policy

Our database write policy is carefully structured to maintain data consistency, ensure performance, and support scalability across our distributed microservices system.

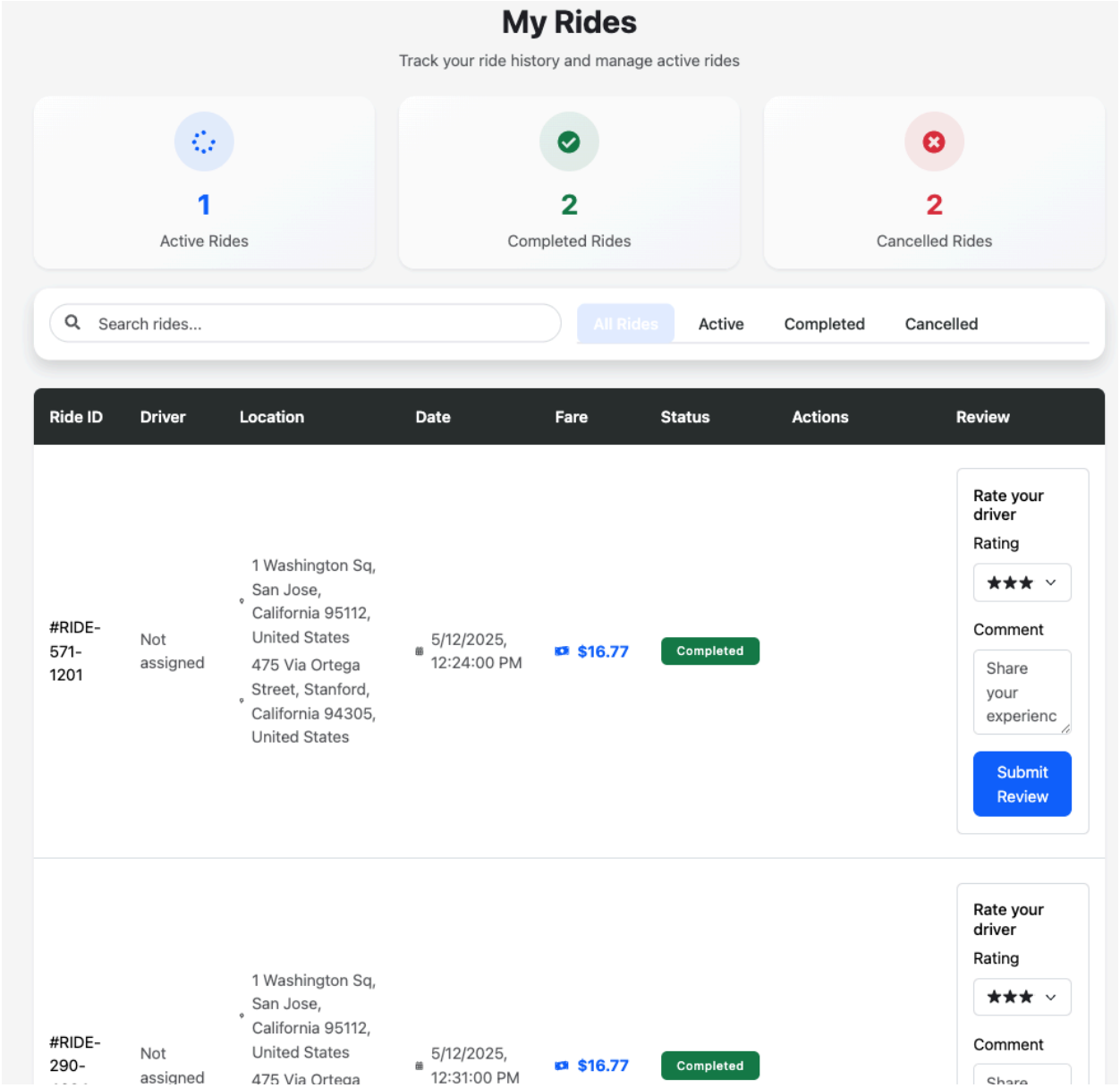
Write Operations:

- **Create Operations**
    - New customer registration
    - New driver registration
    - New ride creation
    - New review submission
- **Update Operations**
    - Customer profile updates
    - Driver status updates
    - Ride status changes
    - Rating updates
- **Write Policies**
    - Atomic operations for data consistency
    - Proper validation before writes
    - Error handling and rollback mechanisms
    - Timestamp tracking for all operations

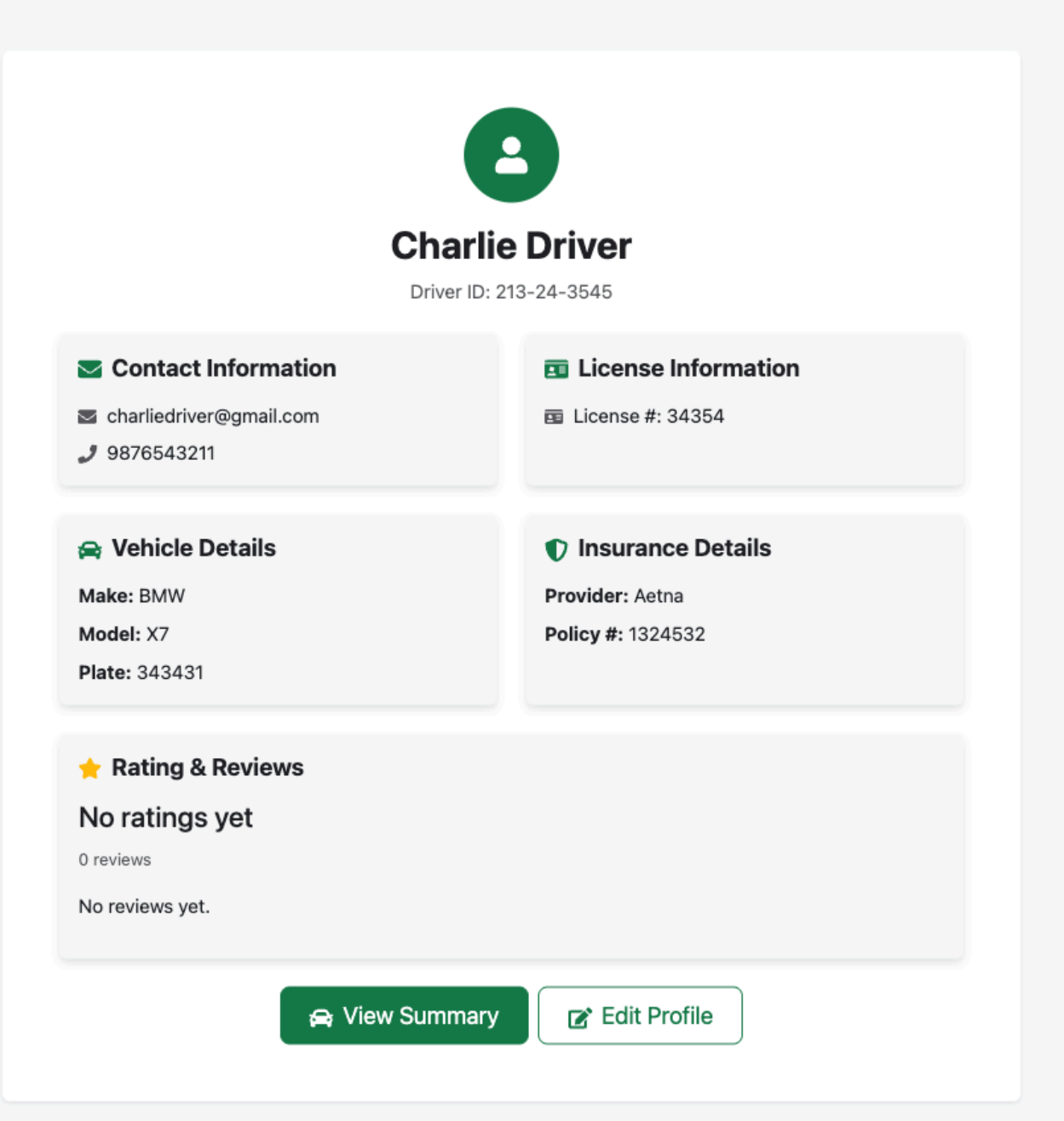
Database Schema - Mermaid Schema Diagram



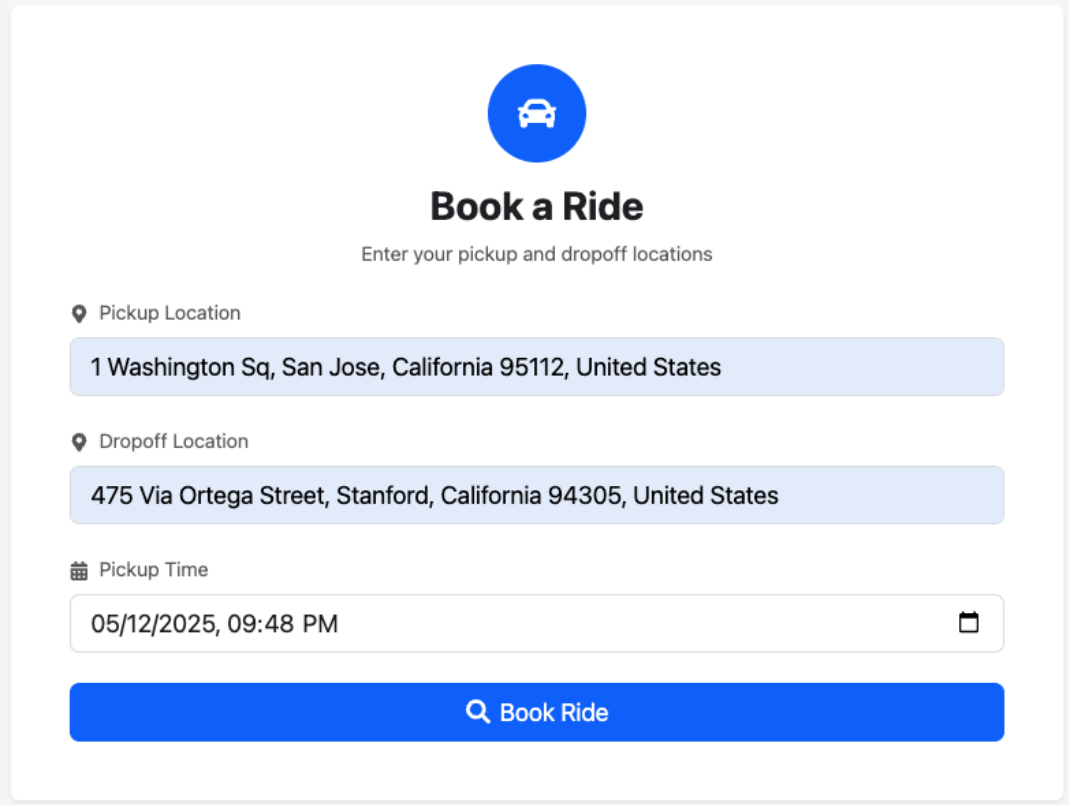
1. **Customer Dashboard**—showing ride booking or ride history



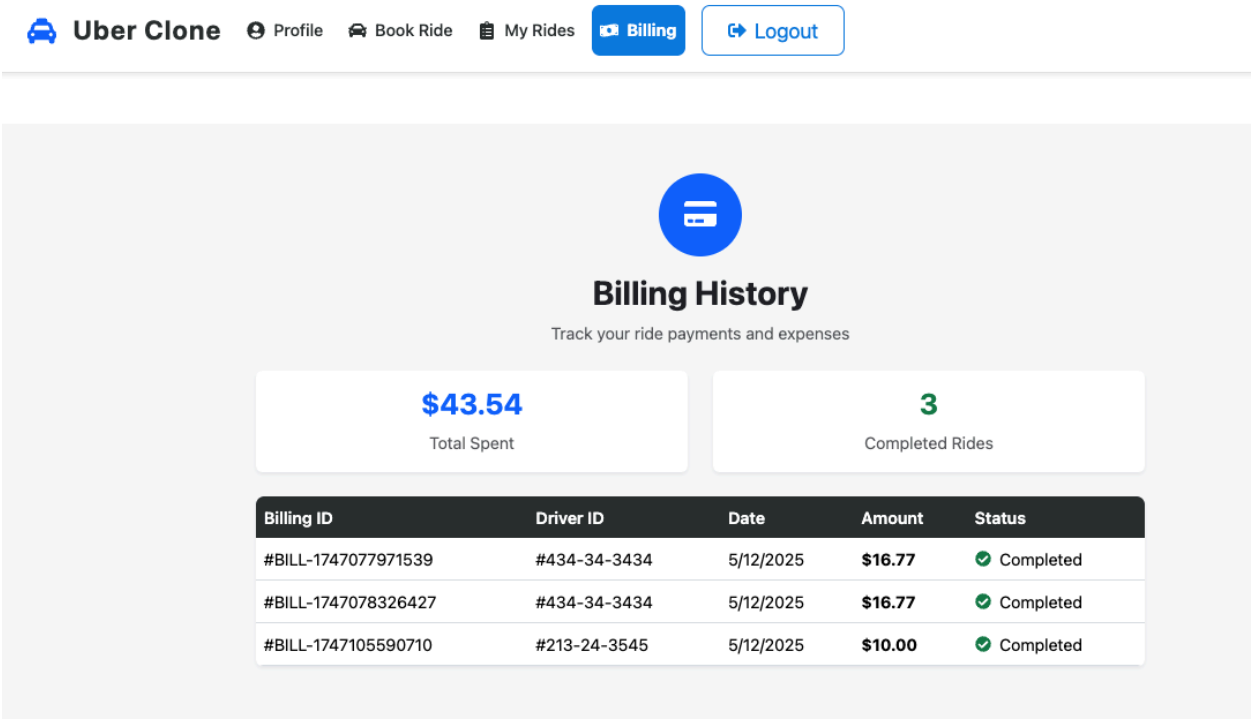
2. **Driver Profile** – Profile



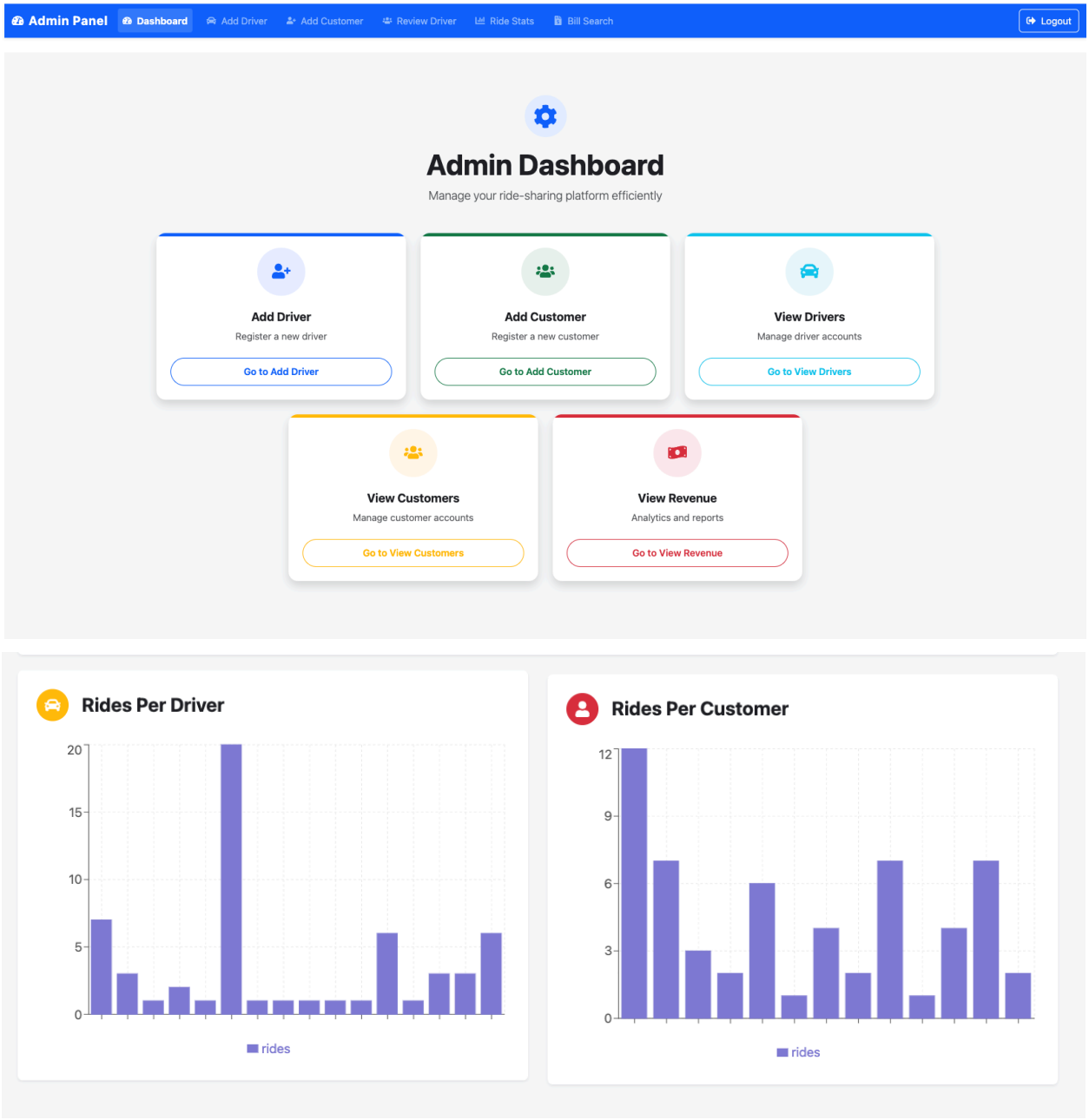
3. **Ride Booking Flow** – map view or ride details



4. **Billing Summary Screen** – showing predicted & actual price



5. **Admin Dashboard** – showing analytics or revenue graph



Observations

Working on this Uber Simulation project helped us understand how a real distributed system works and gave us hands-on experience building a whole application from scratch.

- Splitting the system into different services for drivers, customers, rides, billing, and admin made it easier for each team member to focus on their part. This separation also made debugging and updates more manageable.
- MongoDB worked well for our needs, especially when storing reviews, credit card info, and ride history inside documents. We didn't need to join data often, which made things simpler.
- Kafka helped us connect services without having them rely directly on each other. For example, when a ride was booked, the billing service got the data through Kafka instead of a direct call.
- We used Redis to speed up frequent queries, such as nearby drivers or ride lists. This reduced the time the front end had to wait and lowered pressure on the database.

Lessons Learned

- Input validation is critical. We faced issues early because some services allowed wrong ZIP codes or invalid SSNs.
- Writing into multiple services (like updating a ride and billing simultaneously) can be tricky. We had to ensure things stayed in sync and used Kafka to help.
- Working in a group required clear communication. We had to ensure that the APIs matched on both the front end and the back end. When someone changed something, it affected other parts, too.

- Redis and MongoDB together gave us the performance and flexibility we needed. We learned that performance tuning is something you keep improving as you test more.

This project helped us understand how real backend systems are built and deployed. We also learned how to manage and connect different services.